BFOR516 - Convolutional Neural Networks

Srishti Gupta, Ph.D.

Gradient Descent

Gradient descent is a way to find the best values for a model's parameters (or weights/biases) by minimizing an error function (how wrong the model is).

- We start with some initial values for the weights.
- We calculate how much the error changes (gradient) with respect to these weights.
- We adjust the weights slightly in the opposite direction of the gradient to reduce the error.
- This adjustment is controlled by a small number called the learning rate (η).

$$w(\tau+1)=w(\tau)-\eta\nabla E(w(\tau))$$

- This means we update the weight w by subtracting a small fraction of the gradient.

Gradient Descent

$$w(\tau+1)=w(\tau)-\eta\nabla E(w(\tau))$$

- where:
  - $w(\tau)$ is the current weight.
  - η (eta) is the learning rate, which controls how big a step we take.
  - $\nabla E(w(\tau))$ is the gradient, which tells us the direction and steepness of the slope.
  - $w(\tau+1)$ is the updated weight after taking a step.

Batch vs. Stochastic Gradient Descent

- **Batch Gradient Descent**: Uses the entire dataset to compute the gradient at each step. This ensures accuracy but can be slow for large datasets.
  - $\nabla E(w(\tau))$ is calculated using all training data.
- **Stochastic Gradient Descent (SGD)**: Instead of using the whole dataset, we update the weights based on one data point at a time. This makes the process faster and can help avoid getting stuck in local minima.
  - $\nabla En(w(\tau))$ is calculated using **only one** training example.
- **Mini-batch Gradient Descent**: A middle ground where a small batch of data points is used for each update.
  - $\nabla EB(w(\tau))$ is calculated over a small batch of data points.

Batch vs. Stochastic Gradient Descent

Method	Speed	Stability	Best For
Batch Gradient Descent	Slow	Very stable	Small datasets, high accuracy needs
Stochastic Gradient Descent (SGD)	Fast	Noisy updates	Large datasets, real-time learning
Mini-Batch Gradient Descent	Medium	Balanced	Most practical machine learning problems

Regularization in Neural Networks

- Neural networks are powerful tools for making predictions, but they can sometimes become too complex and start overfitting.
- Regularization helps prevent overfitting by keeping the model's complexity in check.

Regularization - Choosing the Right Number of Hidden Units (M)

- A neural network consists of input units, hidden units, and output units.
- The number of input and output units is determined by the dataset.
- The number of hidden units (M) is a free parameter we can adjust to improve performance.
- Too few hidden units (low M) → The model might be too simple and unable to capture patterns (underfitting).
- Too many hidden units (high M) → The model might become too complex and memorize the data (overfitting).

To find the best value of M:

- We try different values of M and check the validation error (error on unseen data).
- The best M is the one that gives the lowest validation error.

Controlling Complexity with Regularization

Instead of adjusting M, another way to control complexity is by adding a **regularization term** to the error function - **Weight Decay Regularization (L2 Regularization) -** we modify the error function:

$$E_{reg}(w)=E(w) + \lambda/2 * w^T w$$

- $E(w) \rightarrow$ Original error function (sum of squared errors).
- $w^T w \rightarrow$ Sum of squared weights (L2 norm of the weights).
- $\lambda \rightarrow$ Regularization parameter that controls the strength of the regularization.

How it works:

- **If λ is too small**, the network might overfit.
- **If λ is too large**, the network might underfit because it forces the weights to be very small.
- Large weights in a neural network allow it to memorize specific details in the training data, which leads to overfitting.
- Regularization penalizes large weights, encouraging the network to find simpler and more general patterns.

Consistent Gaussian Priors & Advanced Regularization

A simple weight decay approach treats all weights and biases the same way. However, in some cases, we want different weights to have different levels of regularization.

- If input data undergoes transformations (e.g., scaling), the weights should change accordingly.
- A better regularizer considers different sets of weights separately, leading to an improved formula:

$$\lambda_1/2 = \sum_{w \in W_1} w^2 + \lambda_2/2 * \sum_{w \in W_2} w^2$$

where:

- $W_1 \rightarrow$ Weights in the first layer.
- $W_2 \rightarrow$ Weights in the second layer.
- Different λ_1 and λ_2 control regularization separately for each layer.

This approach is more **scale-invariant**, meaning it remains valid even if we change the scale of input or output variables.

Early Stopping as an Alternative to Regularization

Another way to prevent overfitting is early stopping. Instead of adding a regularization term, we monitor the validation error during training:

1. At the beginning, the network is underfitting (it hasn't learned enough).
2. As training progresses, the network improves, and validation error decreases.
3. If we keep training for too long, the network starts overfitting, and validation error increases.

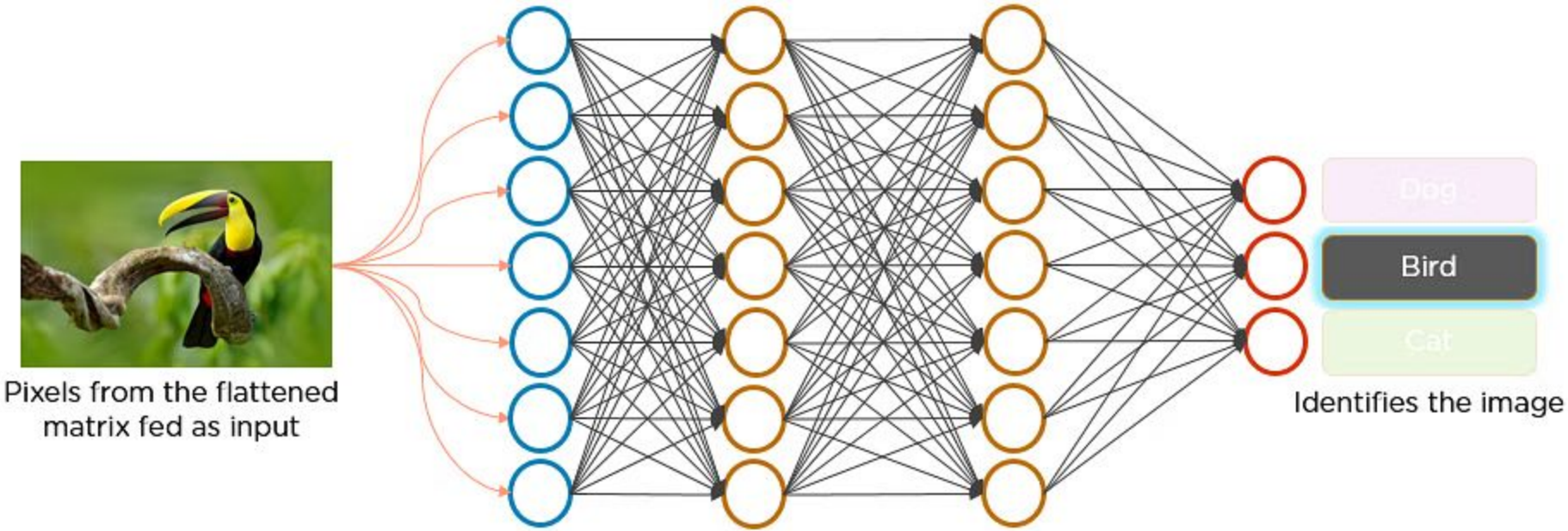
Solution: Stop training when validation error is at its lowest point (before overfitting starts).

This approach is computationally efficient and avoids the need to tune a regularization parameter λ

Convolutional Neural Network (CNN)

- CNN is a class of deep neural networks commonly used in computer vision tasks such as image recognition, object detection, and segmentation.
- CNNs are designed to automatically and adaptively learn spatial hierarchies of features from input images through convolutional layers, pooling layers, and fully connected layers.

CNN Architecture



CNN Architecture

1. **Convolution Layer:**
   - Each unit (or neuron) in this layer detects a feature (like a line, edge, or corner) in the image. All neurons in a feature map share the same weights.
   - A feature map might consist of, for example, 100 neurons arranged in a 10x10 grid. Each neuron processes a small 5x5 patch of the image, and all neurons in the feature map share the same 5x5 weights. This reduces the number of parameters the network needs to learn and makes it more efficient.

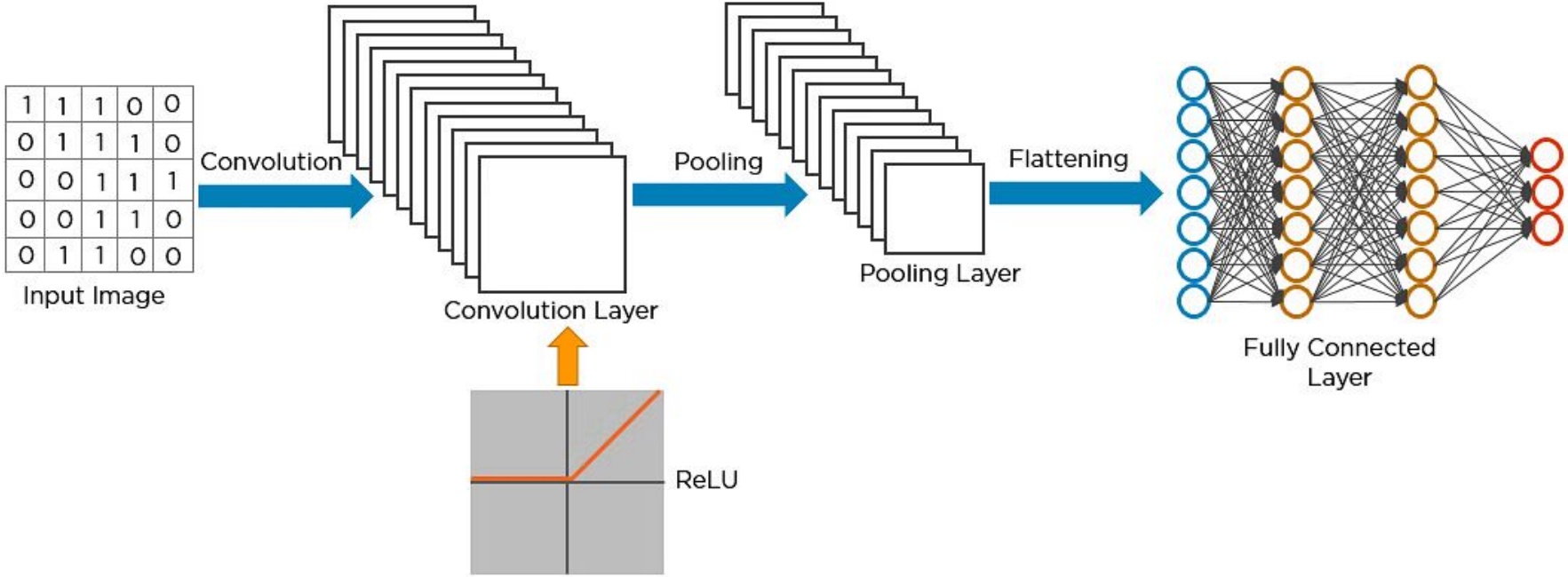
CNN Architecture

2. **Subsampling (Pooling) Layer:**
   - This layer reduces the spatial resolution of the image (for example, turning a 10x10 grid into a 5x5 grid). The idea is that after detecting features with convolution, the network should learn to recognize higher-level features that are less sensitive to small shifts or distortions.
   - This is done by pooling (averaging or taking the maximum of small regions of the feature map).

CNN Architecture

3. **Fully Connected Layer:**
   - After several layers of convolutions and pooling, the network might have a fully connected layer where all the neurons connect to each other, similar to traditional neural networks. This layer is used to make the final prediction (e.g., classifying an image into one of several categories).
4. **Training with Backpropagation:**
   - The CNN is trained using backpropagation, which adjusts the weights of the network based on the error in the prediction. During training, the network learns to detect patterns and refine its weights to improve accuracy.

CNN Architecture



Why CNNs Work Well for Images

- Local patterns like edges, corners, and textures are important in images. CNNs are designed to automatically learn and detect these patterns using the convolutional layers.
- CNNs are translation-invariant, meaning they can recognize an object regardless of its position in the image. This is achieved through the weight sharing mechanism.
- The subsampling layers help make CNNs robust to small distortions like shifts or rotations.